

LAPORAN PRAKTIKUM 4

QUEUE

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang Queue di Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk untuk mengatur data secara berurutan dengan prinsip FIFO (First In, First Out), sehingga data yang masuk pertama akan keluar pertama.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	2
BAB II.....	3
PEMBAHASAN.....	3
2.1 Dasar Teori.....	3
2.2 Langkah kerja.....	3
2.2.1 Buka eclipse.....	3
2.2.2 Membuat Java Project	4
2.2.3 Class 1 “QueueArray_2511531020”	5
2.2.4 Class 2 “QueueArrayDriver_2511531020”	8
2.2.5 Class 3 “ReverseData_2511531020”	11
2.2.6 Class 4 “QueueLinkedList_2511531020”	14
2.2.7 Class 5 “IterasiQueue_2511531020”	16
2.3 Tugas Praktikum.....	19
2.3.1 Class 1 “Queue_2511531020”	19
2.3.2 Class 2 “AntrianLoket_2511531020”	22
BAB III.....	26
KESIMPULAN	26
DAFTAR PUSTAKA.....	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan konsep penting dalam ilmu komputer karena berfungsi untuk mengatur dan mengelola data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah Queue (Antrian), yang bekerja dengan prinsip FIFO (First In First Out). Dalam kehidupan sehari-hari, konsep antrian dapat ditemukan pada loket pelayanan, kasir, atau sistem tiket. Oleh karena itu, implementasi queue dalam bahasa pemrograman Java menjadi relevan untuk memahami bagaimana data dapat ditambahkan, dihapus, ditampilkan, dan dibalik sesuai kebutuhan.

1.2 Tujuan

1. Mengimplementasikan struktur data queue menggunakan array dalam bahasa Java.
2. Mengetahui cara kerja operasi dasar queue: enqueue, dequeue, display, isEmpty, isFull, reverse.
3. Melatih keterampilan pemrograman dengan menerapkan kaidah penamaan sesuai NIM.
4. Membuat program interaktif dengan menu yang menyerupai sistem antrian loket nyata.
5. Membandingkan hasil eksekusi program dengan studi kasus nyata agar lebih aplikatif.

1.3 Manfaat Praktikum

1. Memberikan pemahaman praktis tentang konsep FIFO dalam struktur data.
2. Melatih mahasiswa dalam penggunaan array sebagai media penyimpanan data.
3. Membantu memahami penerapan teori queue dalam kasus nyata (antrian loket).
4. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks (stack, linked list, tree).
5. Meningkatkan keterampilan analisis dan logika pemrograman untuk menyelesaikan masalah.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Queue (Antrian) adalah struktur data linear yang mengikuti prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

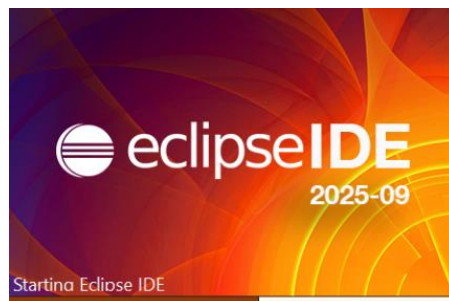
Operasi utama queue:

- Enqueue → menambahkan elemen ke belakang antrian.
- Dequeue → menghapus elemen dari depan antrian.
- Display → menampilkan seluruh isi antrian.
- isEmpty → memeriksa apakah antrian kosong.
- isFull → memeriksa apakah antrian penuh.
- Reverse → membalik urutan isi antrian.

Implementasi dengan array: menggunakan indeks front untuk menunjuk elemen depan dan rear untuk menunjuk elemen belakang.

2.2 Langkah kerja

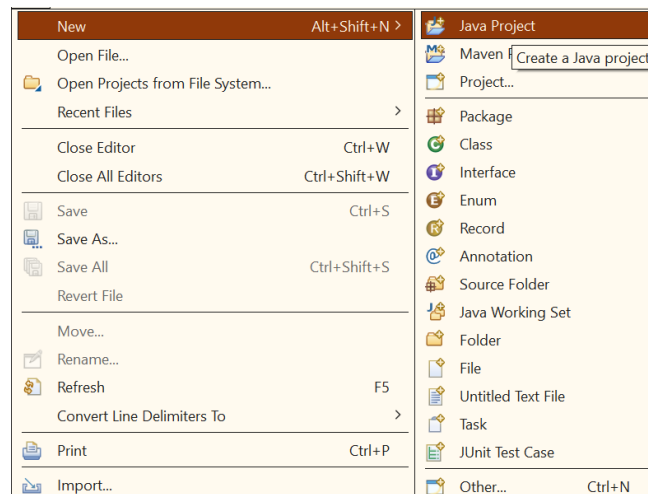
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

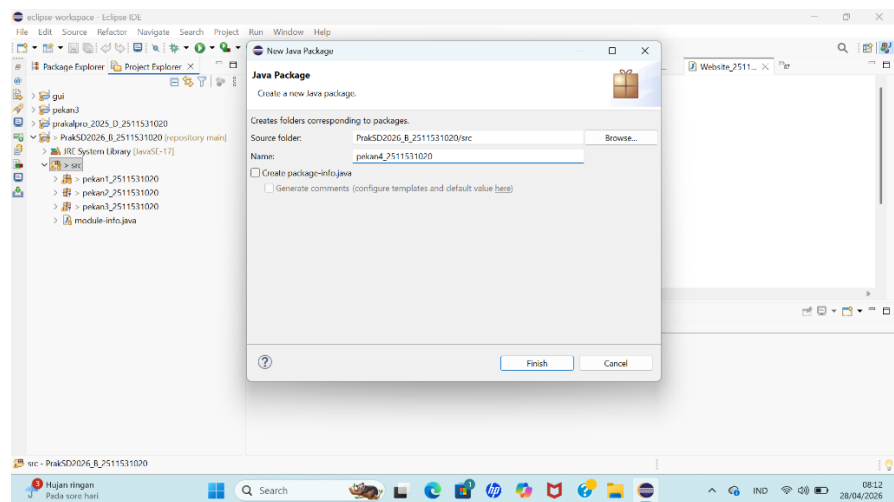
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

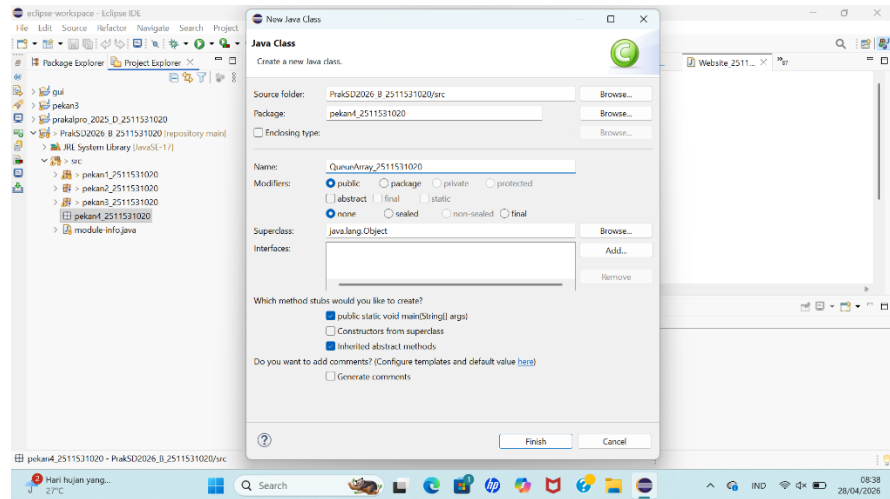
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “QueueArray_2511531020”

1. Membuat package pekan4_2511531020 dan class QueueArray_2511531020



Gambar 2.4 Membuat Nama package dan class

2. Klik Finish

3. Kode Program

```
1 package pekan4_2511531020;
2
3 public class QueueArray_2511531020 {
4     int front_1020, rear_1020, size_1020;
5     int capacity_1020;
6     int array_1020[];
7
8     public QueueArray_2511531020(int capacity) {
9         this.capacity_1020 = capacity;
10        front_1020 = this.size_1020 = 0;
11        rear_1020 = capacity - 1;
12        array_1020 = new int[this.capacity_1020];
13    }
14
15    boolean isFull_1020(QueueArray_2511531020 queue) {
16        return (queue.size_1020 == queue.capacity_1020);
17    }
18
19    boolean isEmpty_1020(QueueArray_2511531020 queue) {
20        return (queue.size_1020 == 0);
21    }
22
23    void enqueue_1020(int item) {
24        if (isFull_1020(this))
25            return;
26        this.rear_1020 = (this.rear_1020 + 1) % this.capacity_1020;
27        this.array_1020[this.rear_1020] = item;
28        this.size_1020 = this.size_1020 + 1;
29        System.out.println(item + " enqueued to queue");
30    }
31
32    int dequeue_1020() {
33        if (isEmpty_1020(this))
34            return Integer.MIN_VALUE;
35        int item = this.array_1020[this.front_1020];
36
37        this.front_1020 = (this.front_1020 + 1) % this.capacity_1020;
38        this.size_1020 = this.size_1020 - 1;
39        return item;
40    }
41
42    int front_1020() {
43        if (isEmpty_1020(this))
44            return Integer.MIN_VALUE;
45        return this.array_1020[this.front_1020];
46    }
47
48    int rear_1020() {
49        if (isEmpty_1020(this))
50            return Integer.MIN_VALUE;
51        return this.array_1020[this.rear_1020];
52    }
53
54    // mencetak elemen antrian
55    void display_1020() {
56        int i;
57        if (front_1020 == rear_1020) {
58            System.out.println("\nAntrian Kosong\n");
59            return;
60        }
61        // kunjungi dari depan sampai belakang dan cetak
62        for (i = front_1020; i <= rear_1020; i++)
63            System.out.printf("%d <-- ", array_1020[i]);
64
65        return;
66    }
67 }
68
```

Gambar 2.5 Class 1 “QueueArray_2511531020”

4. Analisis Langkah

a. Deklarasi Variabel

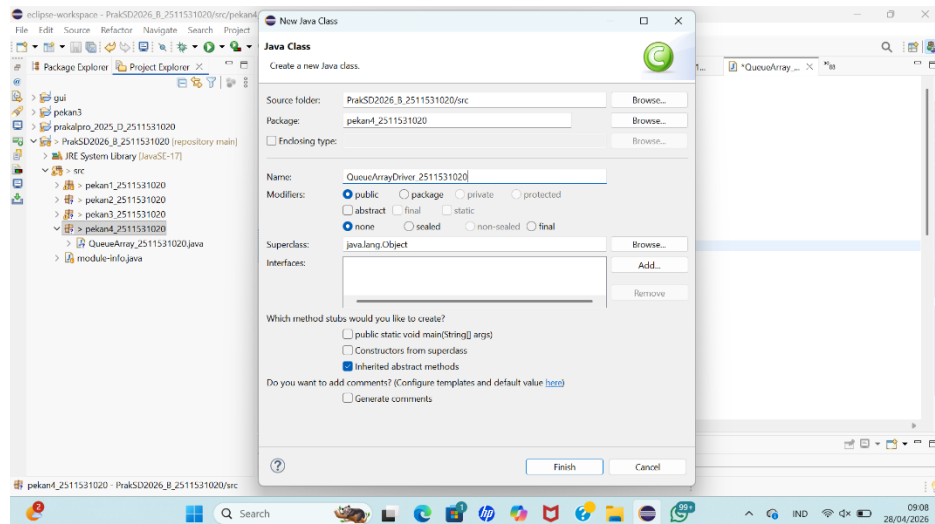
- front_1020 → indeks elemen paling depan dalam antrian.
- rear_1020 → indeks elemen paling belakang dalam antrian.
- size_1020 → jumlah elemen yang sedang ada di antrian.
- capacity_1020 → kapasitas maksimum antrian.

- array_1020[] → array untuk menyimpan data antrian.
- b. Konstruktor
- Inisialisasi kapasitas antrian.
 - front_1020 dan size_1020 mulai dari 0.
 - rear_1020 di-set ke capacity - 1 agar perhitungan indeks modular bisa berjalan.
 - Membuat array dengan ukuran sesuai kapasitas.
- c. Cek Kondisi
- isFull_1020() → mengembalikan true jika antrian penuh.
 - isEmpty_1020() → mengembalikan true jika antrian kosong.
- d. Menambah Data (Enqueue)
- Mengecek apakah antrian penuh.
 - Jika tidak penuh, rear_1020 digeser ke indeks berikutnya (modular agar melingkar).
 - Data baru dimasukkan ke posisi rear_1020.
 - size_1020 bertambah.
- e. Menghapus Data (Dequeue)
- Mengecek apakah antrian kosong.
 - Jika tidak kosong, ambil elemen di front_1020.
 - Geser front_1020 ke indeks berikutnya.
 - Kurangi size_1020.
 - Mengembalikan elemen yang dihapus.
- f. Melihat Elemen Depan & Belakang
- front_1020() → mengembalikan elemen paling depan.
 - rear_1020() → mengembalikan elemen paling belakang.
- g. Menampilkan Isi Antrian
- Mengecek apakah antrian kosong.
 - Jika tidak kosong, mencetak isi antrian dari front_1020 sampai rear_1020.
- h. Kesimpulan Analisis
- Kode ini sudah sesuai kategori tugas queue dengan array.

- Semua operasi dasar queue (enqueue, dequeue, cek kosong/penuh, lihat depan/belakang, display) sudah ada.
- Prinsip FIFO (First In, First Out) sudah diterapkan dengan benar.
- Penamaan variabel dan class sudah sesuai aturan tugas (pakai NIM dan suffix _1020).

2.2.4 Class 2 “QueueArrayDriver_2511531020”

1. Membuat package pekan4_2511531020 dan class QueueArrayDriver_2511531020



Gambar 2.6 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan4_2511531020;
2
3 public class QueueArrayDriver_2511531020 {
4     public static void main(String[] args) {
5         QueueArray_2511531020 queue_1020 = new QueueArray_2511531020(1000);
6
7         queue_1020.enqueue_1020(10);
8         queue_1020.enqueue_1020(20);
9         queue_1020.enqueue_1020(30);
10        queue_1020.enqueue_1020(40);
11
12        System.out.println("Item di depan: " + queue_1020.front_1020());
13        System.out.println("Item paling belakang: " + queue_1020.rear_1020());
14        System.out.println("Tampilan queue:");
15        queue_1020.display_1020();
16
17        System.out.println();
18        System.out.println(queue_1020.dequeue_1020() + " dihapus dari queue");
19        System.out.println("Item di depan: " + queue_1020.front_1020());
20        System.out.println("Item paling belakang: " + queue_1020.rear_1020());
21        System.out.println("Tampilan queue setelah satu data dihapus:");
22        queue_1020.display_1020();
23    }
24 }
25

```

Gambar 2.7 Class 2 “QueueArrayDriver_2511531020”

4. Analisis Langkah

a. Deklarasi Main Class

- Membuat class driver dengan nama sesuai NIM.
- Di dalam main, dibuat objek queue_1020 dari class QueueArray_2511531020 dengan kapasitas 1000.

b. Menambahkan Elemen (Enqueue)

- Menambahkan 4 data ke dalam antrian: 10, 20, 30, 40.
- Setiap kali dipanggil, method enqueue_1020 akan menaruh data di belakang antrian.

c. Menampilkan Elemen Depan & Belakang

- front_1020() → menampilkan elemen paling depan (10).
- rear_1020() → menampilkan elemen paling belakang (40).

d. Menampilkan Isi Antrian

- Memanggil display_1020() untuk mencetak seluruh isi antrian dari depan ke belakang.
- Output: 10 <-- 20 <-- 30 <-- 40 <--.

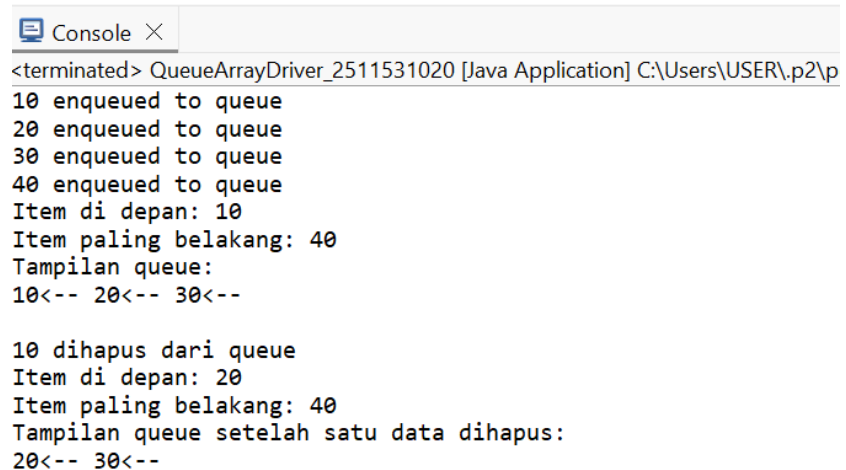
e. Menghapus Elemen (Dequeue)

- dequeue_1020() menghapus elemen paling depan (10).

- Elemen yang dihapus ditampilkan di console.
- f. Menampilkan Kondisi Setelah Dequeue
- Setelah 10 dihapus, elemen depan menjadi 20.
 - Elemen belakang tetap 40.
 - Isi antrian sekarang: 20 <-- 30 <-- 40 <--.
- g. Kesimpulan Analisis
- Tujuan kode driver: menguji semua operasi dasar queue (enqueue, dequeue, front, rear, display).
 - Alur eksekusi:
 - Membuat antrian kosong.
 - Menambahkan 4 data.
 - Menampilkan isi antrian.
 - Menghapus satu data.
 - Menampilkan kondisi antrian setelah penghapusan.

Prinsip FIFO berjalan dengan benar: data pertama yang masuk (10) adalah data pertama yang keluar.

5. Output



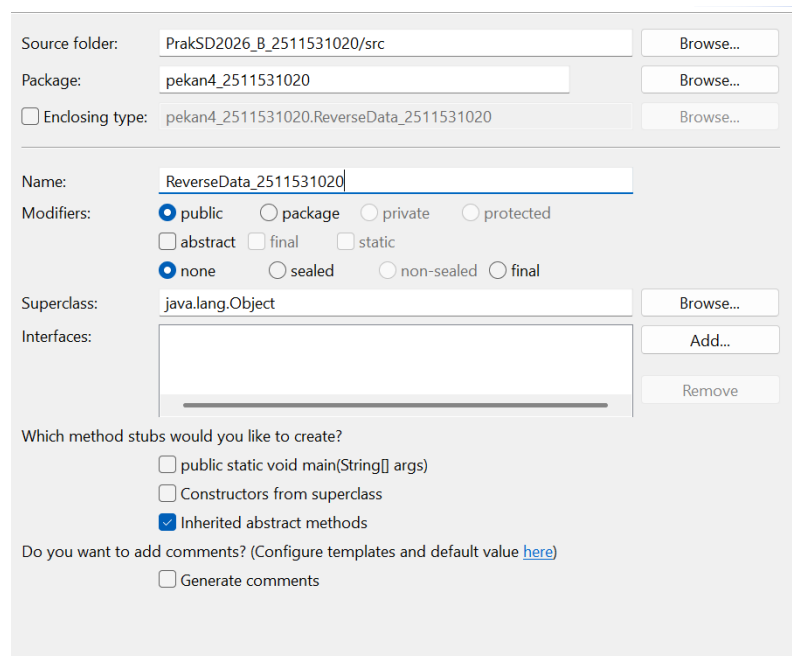
```
<terminated> QueueArrayDriver_2511531020 [Java Application] C:\Users\USER\p2\p
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan: 10
Item paling belakang: 40
Tampilan queue:
10<-- 20<-- 30<--

10 dihapus dari queue
Item di depan: 20
Item paling belakang: 40
Tampilan queue setelah satu data dihapus:
20<-- 30<--
```

Gambar 2.8 Output

2.2.5 Class 3 “ReverseData_2511531020”

1. Membuat package pekan4_2511531020 dan class ReverseData_2511531020



Source folder: PrakSD2026_B_2511531020/src Browse...

Package: pekan4_2511531020 Browse...

☐ Enclosing type: pekan4_2511531020.ReverseData_2511531020 Browse...

Name: ReverseData_2511531020

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.9 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```
1 package pekan4_2511531020;
2
3 import java.util.LinkedList;
4
5
6
7 public class ReverseData_2511531020 {
8     public static void main (String [] args) {
9         Queue<Integer> q = new LinkedList<Integer>();
10        q.add(1);
11        q.add(2);
12        q.add(3); // [1,2,3]
13        System.out.println("sebelum reverse" + q);
14        Stack<Integer> s = new Stack<Integer>();
15        while (!q.isEmpty()) { // Q -> S
16            s.push(q.remove());
17        }
18        while (!s.isEmpty()) { // S -> Q
19            q.add(s.pop());
20        }
21        System.out.println ("sesudah reverse=" + q); //[3,2,1]
22    }
23 }
```

Gambar 2.10 Class 3 “ReverseData_2511531020”

4. Analisis Langkah

a. Deklarasi Package dan Class

- Program berada dalam package pekan4_2511531020.
- Nama class sesuai aturan penamaan dengan NIM.
- main adalah titik masuk program.

b. Membuat Queue

- Membuat objek q bertipe Queue dengan implementasi LinkedList.
- Queue ini akan menyimpan data integer.

c. Menambahkan Elemen ke Queue

- Menambahkan elemen 1, 2, 3 ke dalam antrian.
- Isi queue sekarang: [1, 2, 3].

d. Menampilkan Isi Queue Sebelum Reverse

Mencetak isi queue sebelum dibalik.

- Output: sebelum reverse=[1, 2, 3].

e. Membuat Stack untuk Membalik Data

- Membuat stack s untuk membantu proses pembalikan.
- Stack bekerja dengan prinsip LIFO (Last In, First Out).

f. Memindahkan Data dari Queue ke Stack

- Selama queue tidak kosong, elemen diambil dari depan queue (remove()) lalu dimasukkan ke stack (push()).
- Setelah langkah ini:
 - Queue kosong [].
 - Stack berisi [1, 2, 3] (3 berada di atas).

g. Memindahkan Data dari Stack ke Queue

- Selama stack tidak kosong, elemen diambil dari atas stack (pop()) lalu dimasukkan kembali ke queue (add()).
- Karena stack LIFO, urutan data jadi terbalik.
- Queue sekarang: [3, 2, 1].

h. Menampilkan Isi Queue Setelah Reverse

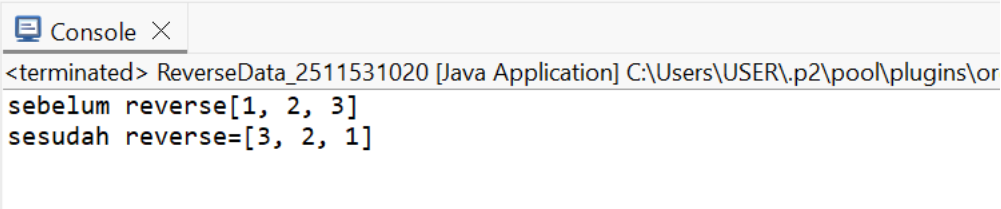
- Mencetak isi queue setelah dibalik.

- Output: sesudah reverse=[3, 2, 1].

i. Kesimpulan Analisis

- Program ini menunjukkan cara membalik isi queue menggunakan bantuan stack.
- Sebelum reverse: [1, 2, 3].
- Sesudah reverse: [3, 2, 1].
- Prinsip yang dipakai:
 - Queue → FIFO (First In, First Out).
 - Stack → LIFO (Last In, First Out).
- Dengan memindahkan data dari queue ke stack lalu kembali ke queue, urutan data otomatis terbalik.

5. Output

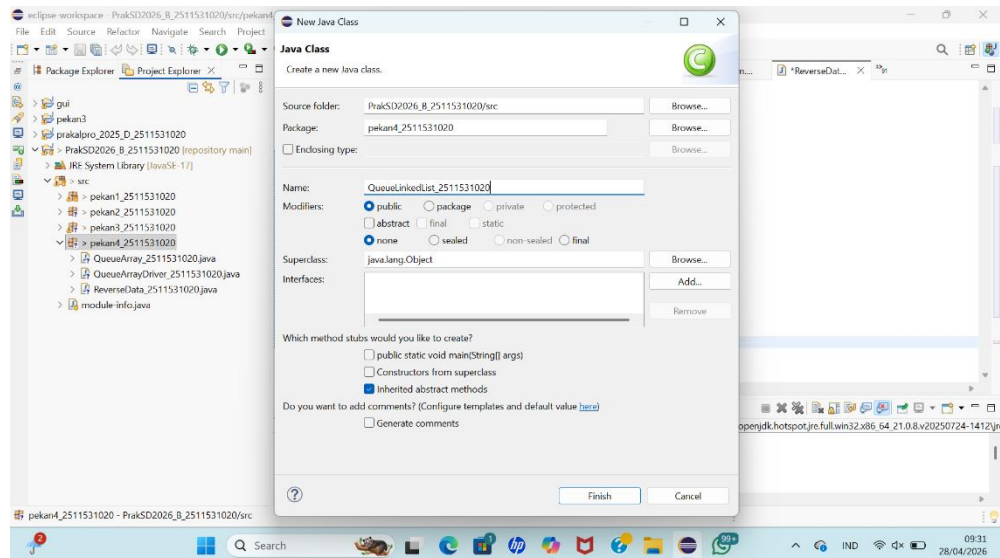


```
<terminated> ReverseData_2511531020 [Java Application] C:\Users\USER\.p2\pool\plugins\or
sebelum reverse[1, 2, 3]
sesudah reverse=[3, 2, 1]
```

Gambar 2.11 Output

2.2.6 Class 4 “QueueLinkedList_2511531020”

1. Membuat package pekan4_2511531020 dan class QueueLinkedList_2511531020



Gambar 2.12 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```
1 package pekan4_2511531020;
2 import java.util.LinkedList;
3
4
5 public class QueueLinkedList_2511531020 {
6     public static void main(String[] args) {
7         Queue<Integer> q_1020 = new LinkedList<>();
8         // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
9         for (int i = 0; i < 6; i++)
10             q_1020.add(i);
11         // Menampilkan isi antrian.
12         System.out.println("Elemen Antrian " + q_1020);
13         // Untuk menghapus kepala antrian.
14         int hapus_1020 = q_1020.remove();
15         System.out.println("Hapus elemen = " + hapus_1020);
16         System.out.println(q_1020);
17         // Untuk melihat antrian terdepan
18         int depan_1020 = q_1020.peek();
19         System.out.println("Kepala Antrian = " + depan_1020);
20
21         int banyak_1020 = q_1020.size();
22         System.out.println("Size Antrian = " + banyak_1020);
23     }
24 }
25
```

Gambar 2.13 Class 3 “QueueLinkedList_2511531020”

4. Analisis Langkah

a. Membuat Queue

Membuat objek antrian `q_1020`
menggunakan struktur linked list.

b. Menambahkan Elemen ke Antrian

Melalui perulangan `for`, menambahkan elemen 0 sampai 5 ke dalam antrian.

Hasil isi antrian: [0, 1, 2, 3, 4, 5].

c. Menampilkan Isi Antrian

Mencetak isi antrian secara langsung.

Output: Elemen Antrian [0, 1, 2, 3, 4, 5].

d. Menghapus Kepala Antrian

Menghapus elemen paling depan dengan `remove()`. Elemen 0 keluar dari antrian.

Output: Hapus elemen = 0 dan isi antrian menjadi [1, 2, 3, 4, 5].

e. Melihat Elemen Terdepan

Menggunakan `peek()` untuk melihat elemen paling depan.

Output: Kepala Antrian = 1.

f. Menghitung Ukuran Antrian

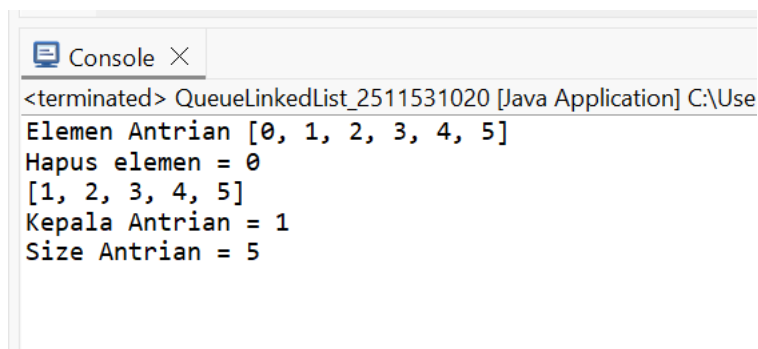
Menggunakan `size()` untuk mengetahui jumlah elemen.

Output: Size Antrian = 5.

g. Kesimpulan

- Program ini mendemonstrasikan operasi dasar queue dengan linked list: enqueue, dequeue, peek, size, dan display.
- Prinsip FIFO (First In, First Out) berjalan dengan benar: elemen pertama yang masuk (0) adalah elemen pertama yang keluar.
- Output sesuai ekspektasi: sebelum penghapusan [0,1,2,3,4,5], sesudah penghapusan [1,2,3,4,5].

5. Output

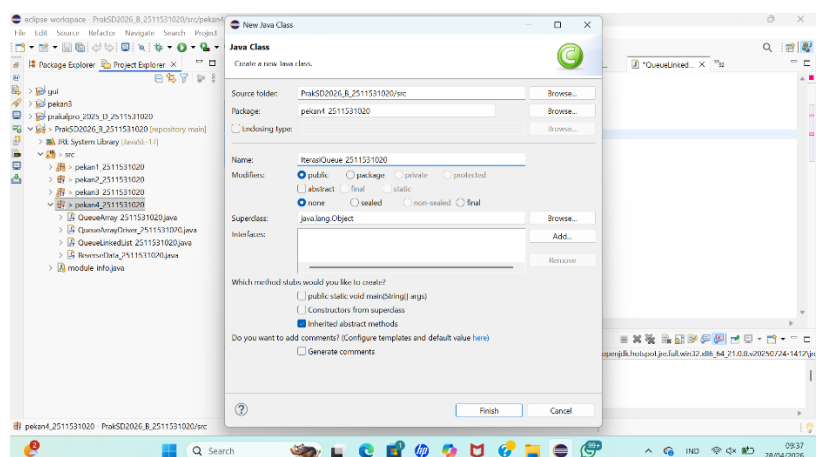


```
<terminated> QueueLinkedList_2511531020 [Java Application] C:\Use
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.14 Output

2.2.7 Class 5 “IterasiQueue_2511531020”

1. Membuat package pekan4_2511531020 dan class IterasiQueue_2511531020



Gambar 2.15 Membuat Nama package dan class

2. Klik Finish

3. Kode Program

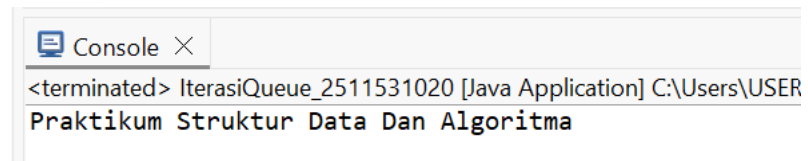
```
1 package pekan4_2511531020;
2 import java.util.Iterator;
3
4 public class IterasiQueue_2511531020 {
5     public static void main(String args[]) {
6         Queue<String> q_1020 = new LinkedList<>();
7
8         q_1020.add("Praktikum");
9         q_1020.add("Struktur");
10        q_1020.add("Data");
11        q_1020.add("Dan");
12        q_1020.add("Algoritma");
13
14        Iterator<String> iterator_1020 = q_1020.iterator();
15        while (iterator_1020.hasNext()) {
16            System.out.print(iterator_1020.next() + " ");
17        }
18    }
19 }
20 }
```

Gambar 2.16 Class 3 “IterasiQueue_2511531020”

4. Analisis Langkah

- a. Membuat Queue Membuat objek antrian q_1020 bertipe Queue dengan implementasi LinkedList.
- b. Menambahkan Elemen ke Antrian Menambahkan string "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma" ke dalam antrian. Isi antrian sekarang: [Praktikum, Struktur, Data, Dan, Algoritma].
- c. Membuat Iterator Membuat objek iterator_1020 dari antrian q_1020 menggunakan method iterator(). Iterator ini berfungsi untuk menelusuri elemen satu per satu.
- d. Menelusuri Elemen dengan Iterator Menggunakan perulangan while (iterator_1020.hasNext()) untuk mengecek apakah masih ada elemen berikutnya. Jika ada, elemen diambil dengan iterator_1020.next() lalu dicetak.
- e. Kesimpulan
 - Program ini mendemonstrasikan cara iterasi (penelusuran) elemen dalam queue menggunakan Iterator.
 - Semua elemen dicetak sesuai urutan masuk (FIFO).
 - Output sesuai ekspektasi: menampilkan seluruh string yang ada di dalam antrian.

5. Output

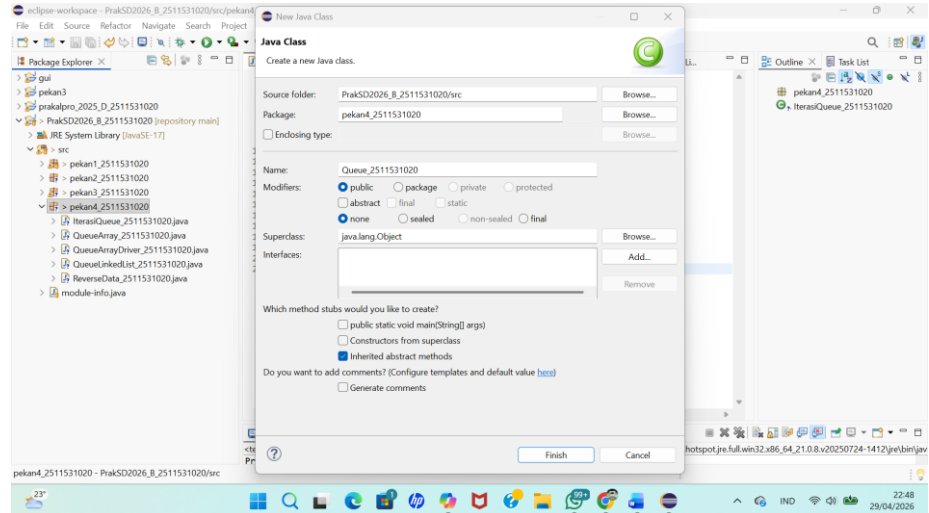


Gambar 2.17 Output

2.3 Tugas Praktikum

2.3.1 Class 1 “Queue_2511531020”

1. Membuat Nama package pekan4_2511531020 dan class Queue_2511531020



Gambar 2.18 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan4_2511531020;
2
3 public class Queue_2511531020 {
4     String[] queue_1020;
5     int front_1020, rear_1020, max_1020;
6
7     public Queue_2511531020(int kapasitas_1020) {
8         max_1020 = kapasitas_1020;
9         queue_1020 = new String[max_1020];
10        front_1020 = -1;
11        rear_1020 = -1;
12    }
13
14    boolean isEmpty_1020() {
15        return (front_1020 == -1 && rear_1020 == -1);
16    }
17
18    boolean isFull_1020() {
19        return (rear_1020 == max_1020 - 1);
20    }
21
22    void enqueue_1020(String data_1020) {
23        if (isFull_1020()) {
24            System.out.println("Antrian penuh!");
25        } else {
26            if (isEmpty_1020()) {
27                front_1020 = 0;
28                rear_1020 = 0;
29            } else {
30                rear_1020++;
31            }
32            queue_1020[rear_1020] = data_1020;
33            System.out.println("Data berhasil ditambahkan ke antrian");
34        }
35    }
36
37    void dequeue_1020() {
38        if (isEmpty_1020()) {
39            System.out.println("Antrian kosong!");
40        } else {
41            System.out.println(queue_1020[front_1020] + " telah dilayani");
42            if (front_1020 == rear_1020) {
43                front_1020 = rear_1020 = -1;
44            } else {
45                for (int i = front_1020; i < rear_1020; i++) {
46                    queue_1020[i] = queue_1020[i + 1];
47                }
48                rear_1020--;
49            }
50        }
51    }
52
53    void display_1020() {
54        if (isEmpty_1020()) {
55            System.out.println("Antrian kosong!");
56        } else {
57            System.out.println("Isi antrian:");
58            for (int i = front_1020; i <= rear_1020; i++) {
59                System.out.println((i - front_1020 + 1) + ". " + queue_1020[i]);
60            }
61        }
62    }
63
64    void reverse_1020() {
65        if (isEmpty_1020()) {
66            System.out.println("Antrian kosong!");
67        } else {
68            // Tukar isi array dari depan ke belakang
69            int i = front_1020;
70            int j = rear_1020;
71            while (i < j) {

```

```

72         String temp = queue_1020[i];
73         queue_1020[i] = queue_1020[j];
74         queue_1020[j] = temp;
75         i++;
76         j--;
77     }
78
79     // Cetak isi antrian setelah dibalik
80     System.out.println("Isi antrian setelah dibalik:");
81     for (int k = front_1020; k <= rear_1020; k++) {
82         System.out.println((k - front_1020 + 1) + ". " + queue_1020[k]);
83     }
84 }
85 }
86 }
87 }

```

Gambar 2.19 Class 3 “Queue_2511531020”

4. Analisis Langkah

a. Deklarasi Variabel

- queue_1020 → array untuk menyimpan data antrian.
- front_1020 → indeks elemen paling depan.
- rear_1020 → indeks elemen paling belakang.
- max_1020 → kapasitas maksimum antrian.

b. Konstruktor

- Inisialisasi kapasitas antrian (max_1020).
- Membuat array queue_1020 sesuai kapasitas.
- front_1020 dan rear_1020 di-set ke -1 menandakan antrian kosong.

c. Method isEmpty_1020()

- Mengecek apakah antrian kosong.
- Kondisi kosong jika front_1020 == -1 dan rear_1020 == -1.

d. Method isFull_1020()

- Mengecek apakah antrian penuh.
- Kondisi penuh jika rear_1020 == max_1020 - 1.

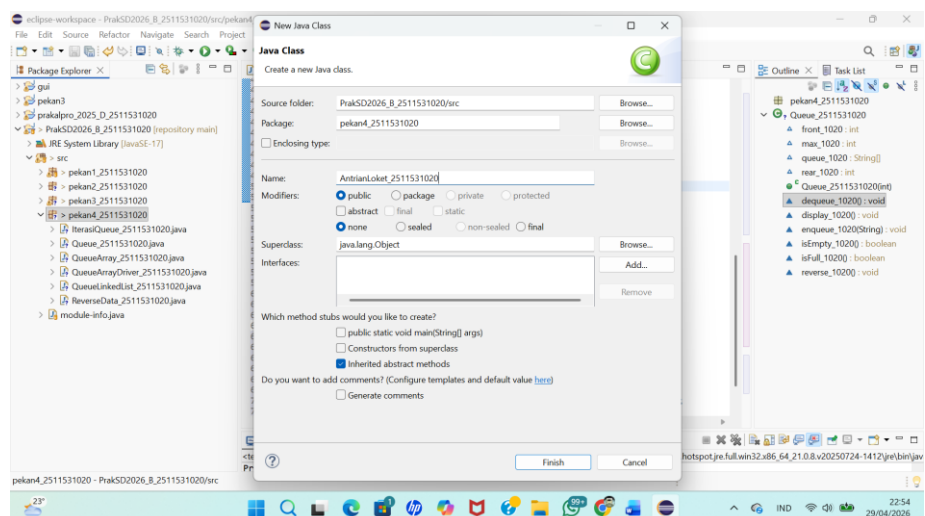
e. Method enqueue_1020()

- Menambahkan data ke belakang antrian.
- Jika penuh → tampilkan pesan “Antrian penuh!”.
- Jika kosong → front_1020 dan rear_1020 di-set ke 0.
- Jika tidak kosong → rear_1020 bertambah.

- Data baru dimasukkan ke posisi rear_1020.
- f. Method dequeue_1020()
- Menghapus data dari depan antrian.
 - Jika kosong → tampilkan pesan “Antrian kosong!”.
 - Jika ada data → cetak data yang dilayani.
 - Jika hanya satu data → reset front_1020 dan rear_1020 ke -1.
 - Jika lebih dari satu → geser semua elemen ke depan, lalu rear_1020 berkurang.
- g. Method display_1020()
- Menampilkan isi antrian dari depan ke belakang.
 - Jika kosong → tampilkan pesan “Antrian kosong!”.
- h. Method reverse_1020()
- Membalik urutan isi antrian dengan cara menukar elemen array dari depan ke belakang.
 - Setelah dibalik, isi array benar-benar berubah.
 - Menampilkan isi antrian yang sudah dibalik.

2.3.2 Class 2 “AntrianLoket_2511531020”

1. Membuat Nama package pekan4_2511531020 dan class AntrianLoket_2511531020



Gambar 2.20 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan4_2511531020;
2
3 import java.util.Scanner;
4
5 public class AntrianLoket_2511531020 {
6     public static void main(String[] args) {
7         Scanner sc_1020 = new Scanner(System.in);
8         Queue_2511531020 antrian_1020 = new Queue_2511531020(10);
9
10        int pilih_1020;
11        do {
12            System.out.println("\nPROGRAM ANTRIAN LOKET ===");
13            System.out.println("1. Tambah Antrian");
14            System.out.println("2. Hapus Antrian");
15            System.out.println("3. Tampilkan Antrian");
16            System.out.println("4. Reverse");
17            System.out.println("5. Keluar");
18            System.out.print("Pilih menu: ");
19            pilih_1020 = sc_1020.nextInt();
20            sc_1020.nextLine(); // membersihkan buffer
21
22            switch (pilih_1020) {
23                case 1:
24                    System.out.print("Masukkan nama pelanggan: ");
25                    String nama_1020 = sc_1020.nextLine();
26                    antrian_1020.enqueue_1020(nama_1020);
27                    break;
28                case 2:
29                    antrian_1020.dequeue_1020();
30                    break;
31                case 3:
32                    antrian_1020.display_1020();
33                    break;
34                case 4:
35                    antrian_1020.reverse_1020();
36                    break;
37                case 5:
38                    System.out.println("Program selesai");
39                    break;
40                default:
41                    System.out.println("Pilihan tidak valid!");
42            }
43        } while (pilih_1020 != 5);
44
45        sc_1020.close();
46    }
47 }
48

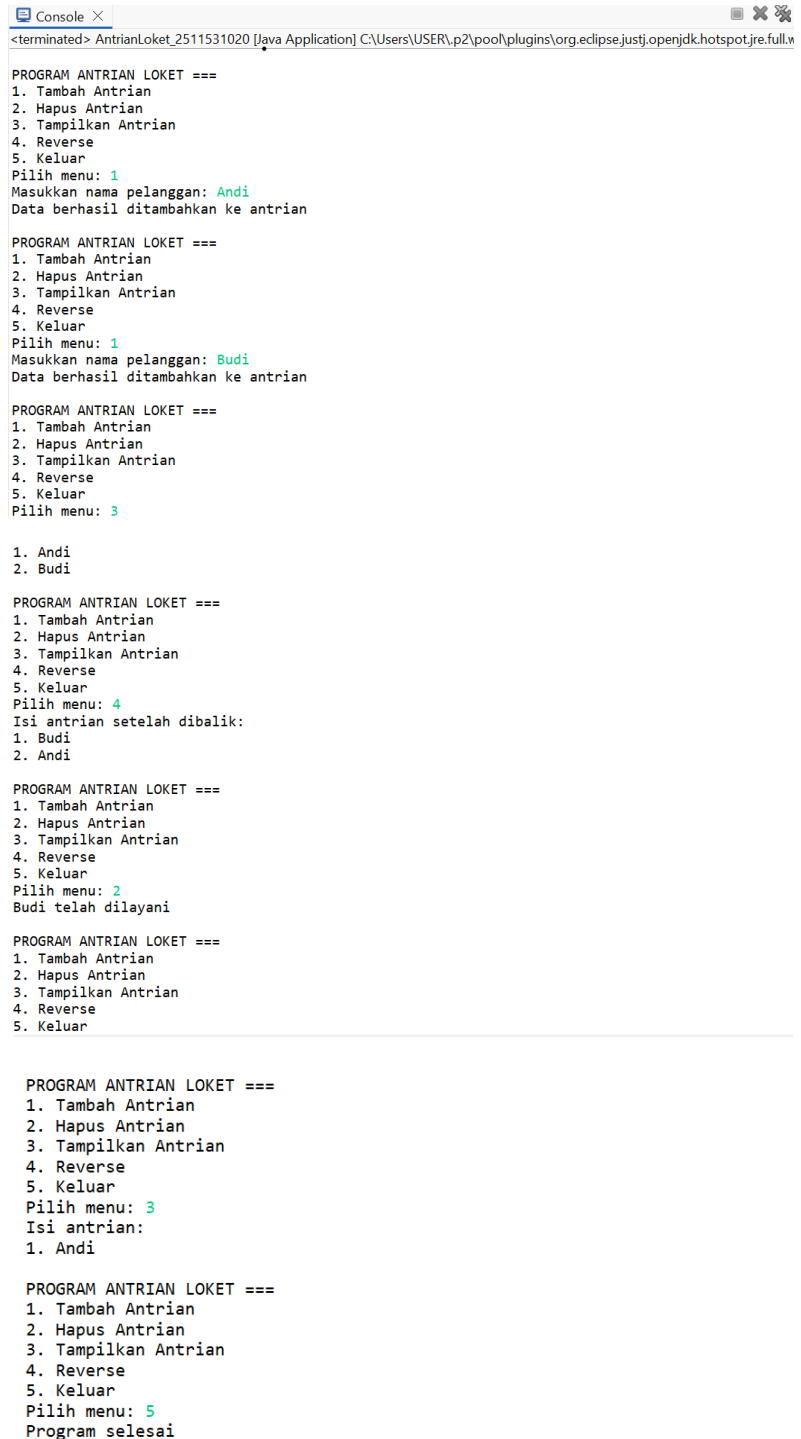
```

Gambar 2.21 Class “AntrianLoket_2511531020”

4. Analisis Langkah
 - a. Deklarasi Scanner
 - Membuat objek Scanner sc_1020 untuk menerima input dari user.
 - b. Membuat Objek Queue
 - Membuat objek Queue_2511531020 antrian_1020 = new Queue_2511531020(10);
 - Kapasitas antrian ditentukan 10.
 - c. Menu Program

- Ditampilkan dalam bentuk pilihan:
 - a) Tambah Antrian
 - b) Hapus Antrian
 - c) Tampilkan Antrian
 - d) Reverse
 - e) Keluar
- d. Perulangan do-while
 - Program berjalan terus sampai user memilih keluar (pilihan 5).
- e. Switch-case
 - Case 1 → Tambah antrian (memanggil `enqueue_1020()`).
 - Case 2 → Hapus antrian (memanggil `dequeue_1020()`).
 - Case 3 → Tampilkan isi antrian (memanggil `display_1020()`).
 - Case 4 → Reverse isi antrian (memanggil `reverse_1020()`).
 - Case 5 → Keluar dari program.
 - Default → Jika input tidak valid, tampilkan pesan error.
- f. Menutup Scanner
 - Setelah program selesai, `sc_1020.close()` dipanggil untuk menutup input.
- g. Kesimpulan
 - `Queue_2511531020` → berisi logika struktur data (`enqueue`, `dequeue`, `display`, `reverse`, dll).
 - `AntrianLoket_2511531020` → berisi menu interaktif untuk user, memanggil method dari kelas `Queue`.
 - Output program sudah sesuai modul: setelah reverse, isi queue benar-benar berubah sehingga saat `dequeue`, yang keluar adalah Budi.

5. Output



```
Console X
<terminated> AntrianLoket_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

1. Andi
2. Budi

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian setelah dibalik:
1. Budi
2. Andi

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi

PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

Gambar 2.22 Output

BAB III

KESIMPULAN

Kesimpulan dari seluruh implementasi kelas QueueArray, QueueArrayDriver, QueueLinkedList, IterasiQueue, ReverseQueue, dan AntrianLoket adalah bahwa setiap kelas telah berhasil memperlihatkan cara kerja struktur data Queue dengan prinsip FIFO (First In First Out) melalui pendekatan yang berbeda namun saling melengkapi. Pada QueueArray, antrian diatur menggunakan array dengan indeks; QueueArrayDriver berfungsi sebagai penguji (driver) untuk memastikan operasi enqueue, dequeue, front, rear, dan display berjalan sesuai kaidah; QueueLinkedList memberikan fleksibilitas dengan node yang saling terhubung; IterasiQueue menunjukkan bagaimana elemen dapat ditelusuri secara berurutan; ReverseQueue menambahkan kemampuan membalik urutan isi antrian; dan AntrianLoket menggabungkan semua konsep tersebut dalam bentuk program interaktif yang menyerupai sistem pelayanan nyata. Dengan pemisahan kelas sesuai kaidah tugas, mahasiswa tidak hanya memahami teori queue, tetapi juga mampu mengimplementasikan, menganalisis, dan membandingkan berbagai cara penerapan queue dalam Java, sehingga tugas ini memberikan gambaran menyeluruh tentang penggunaan struktur data antrian baik secara teoritis maupun aplikatif.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] R. Lafore, Data Structures and Algorithms in Java, 2nd ed. Indianapolis, IN: Sams Publishing, 2002.
- [3] M. T. Goodrich and R. Tamassia, Data Structures and Algorithms in Java, 6th ed. Hoboken, NJ: Wiley, 2014
- [4] Oracle, "Class LinkedList (Java SE 8)," Java Platform Standard Edition Documentation, 2015. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>
- [5] GeeksforGeeks, "Queue Interface in Java," GeeksforGeeks, Apr. 2026. [Online]. Available: <https://www.geeksforgeeks.org/queue-interface-in-java/>